Oral Cancer Detection using a Telegram Bot: Interview Script (about 24 minutes)

Before you start: replace my guesses with your real details

Time plan

Part	Topic	Time
1	Introduction	1 min
2	The problem	1.5 min
3	Why a Telegram bot	1 min
4	System overview	1.5 min
5	Dataset	1.5 min
6	Preprocessing with OpenCV	2 min
7	The model	2.5 min
8	Training in Colab	2 min
9	Evaluation	2.5 min
10	Building the Telegram bot	2.5 min
11	Live demo walk-through	1 min
12	Challenges	1.5 min
13	Limitations	1.5 min
14	Future work	1 min
15	Closing	1 min
	Total	**about 24 min**

THE SCRIPT

Part 1: Introduction (1 min)

Good morning. My name is [Your Name]. I am a final-year B.Tech student in Artificial Intelligence and Data Science.

Today I want to explain my project. It is called **Oral Cancer Detection using a Telegram Bot**.

In this project, I trained a deep learning model to look at a photo of the mouth. Then I connected this model to a Telegram bot.

A user sends a photo to the bot. In a few seconds, the bot replies. It says if the image looks **cancerous or non-cancerous**. It also gives a **confidence score**.

I used Python, TensorFlow and Keras, OpenCV, and Google Colab. I also used the Telegram Bot API.

Part 2: The problem (1.5 min)

Let me first explain the problem.

Oral cancer is one of the common cancers in India. A big reason is tobacco, gutka, and betel nut.

The main issue is late detection. Many people notice a white patch or a sore in the mouth. But they ignore it. By the time they go to a doctor, the cancer has grown.

If oral cancer is found early, treatment works much better. So early screening is very important.

Another issue is access. In villages and small towns, there are few specialists. People may need to travel far just for a first check.

So I asked one question. Can we use AI to give a **quick first screening** from a simple photo?

I want to be clear about one thing. My system is **not a doctor**. It does not give a final diagnosis. It only gives a first signal, so the person can decide to see a doctor sooner.

Part 3: Why a Telegram bot (1 min)

Many students build a model and stop at the notebook. I wanted my model to be easy to use for a normal person.

I chose Telegram for four reasons.

First, almost everyone has a smartphone and a messaging app. Second, the user does not need to install a new app. Third, the Telegram Bot API is free and easy to use. Fourth, sending a photo in a chat is something everyone already knows how to do.

So the user only needs to open the chat and send a photo. That is the whole experience.

Part 4: System overview (1.5 min)

Let me explain the full flow in five simple steps.

Step 1. The user sends an oral cavity image to the bot on Telegram.

Step 2. The bot receives the image through the Telegram Bot API and downloads it.

Step 3. My code uses OpenCV to read the image, resize it, and prepare it for the model.

Step 4. The trained deep learning model looks at the image and gives a probability.

Step 5. The bot sends back the result. It says cancerous or non-cancerous, with a confidence score.

So there are two big parts. One part is the **model**, which I trained in Google Colab. The other part is the **bot**, which connects the model to the user.

Part 5: Dataset (1.5 min)

For the data, I used an image dataset from **Kaggle**. The name is [dataset name].

It has about [N] images in two classes. One class is cancerous images, and the other class is non-cancerous or normal images.

I used Pandas and NumPy to check the data. I counted the images in each class, to see if the data was balanced. [If balanced: The two classes were fairly balanced. If not: One class had more images, so I used augmentation and checked recall carefully.]

Then I split the data into three parts. About 80 percent for **training**, 10 percent for **validation**, and 10 percent for **testing**.

The training set is for learning. The validation set is to check progress and tune settings. The test set is kept hidden until the end, for an honest final score.

I never used the test images during training. This avoids **data leakage**.

Part 6: Preprocessing with OpenCV (2 min)

A model cannot use raw photos directly. The images have different sizes, colours, and lighting. So I did preprocessing.

First, reading the image. I used OpenCV to read each image.

Second, colour fix. OpenCV reads images in BGR order. But my model expects RGB. So I converted BGR to RGB. This is a small step, but if you forget it, the colours are wrong and the accuracy drops.

Third, resizing. I resized every image to 224 by 224 pixels. The model needs the same size every time.

Fourth, normalisation. Pixel values are from 0 to 255. I divided them by 255, so they are between 0 and 1. This helps the model train faster and more stably.

Fifth, data augmentation. My dataset is not very big. A deep model can easily memorise a small dataset. This is called overfitting. To reduce this, I made small random changes to training images. I used rotation, flipping, zoom, and brightness changes.

This also helps in real life. Users will take photos in different light and from different angles. Augmentation prepares the model for that.

I applied augmentation only on the training data. The validation and test data stayed unchanged.

Part 7: The model (2.5 min)

Now the model. I used a **Convolutional Neural Network**, called CNN. A CNN is the best choice for images.

A CNN works in layers. The early layers find simple things like edges and colours. The later layers find complex patterns like texture and shape. For oral cancer, texture and colour changes in the tissue are important clues.

I used **transfer learning**. This means I took a model called **MobileNetV2**, which was already trained on a huge dataset called ImageNet. It already knows how to see edges, textures, and shapes.

Why transfer learning? Because I had a small dataset. Training a big network from zero needs a lot of data. With transfer learning, I got better results with less data and less time.

Here is how I built it.

- I took MobileNetV2 without its last layer.
- I **froze** the base layers, so their knowledge stays safe.
- I added a **Global Average Pooling** layer to reduce the size of the features.
- I added a **Dropout** layer to reduce overfitting.
- I added the final **Dense layer with one neuron and a sigmoid activation**.

The sigmoid gives a number between 0 and 1. This number is the probability that the image is cancerous. If it is above 0.5, the model says cancerous. If it is below 0.5, the model says non-cancerous.

I chose MobileNetV2 because it is light and fast. This matters for a bot, where the user wants the answer in seconds.

For training, I used **binary cross-entropy** as the loss function, because there are two classes. I used the **Adam** optimizer with a small learning rate.

> **If you built a custom CNN instead:** Say: "I built my own CNN with [3] convolution layers, each followed by max pooling, then a flatten layer, a dense layer, dropout, and a final sigmoid layer. I chose a custom CNN to understand every layer."

Part 8: Training in Google Colab (2 min)

I trained the model in **Google Colab**. I chose Colab because it gives a free GPU. Training on my laptop CPU would be very slow.

I connected Colab to Google Drive. I saved the dataset and the trained model there. This is important, because a Colab session can disconnect, and then everything in memory is lost.

I trained for about [20] epochs with a batch size of [32]. One epoch means the model sees all training images once.

During training, I watched two things: the **training loss** and the **validation loss**.

If both go down, the model is learning well. If the training loss goes down but the validation loss goes up, the model is overfitting.

To control this, I used two tools. First, **early stopping**, which stops training when the validation loss does not improve. Second, **model checkpoint**, which saves the best version of the model.

After training, I saved the final model as a [.h5 or .keras] file. The Telegram bot loads this file later.

Part 9: Evaluation (2.5 min)

After training, I tested the model on the **test set**, which it had never seen.

I did not look at accuracy alone. I used four things: **accuracy, precision, recall, and F1-score**. I also made a **confusion matrix**.

My results were: accuracy [***] percent, precision [***] percent, and recall [___] percent.

Let me explain why recall is the most important one for me.

In a cancer screening system, there are two types of mistakes. In the first mistake, the model says "cancerous" for a healthy person. This is a **false positive**. The person gets worried and visits a doctor. The doctor finds nothing. That is not good, but it is safe.

In the second mistake, the model says "non-cancerous" for a person who has cancer. This is a **false negative**. The person feels safe and delays treatment. This is dangerous.

So I focused on **high recall** for the cancer class. Recall tells us: out of all real cancer cases, how many did the model catch?

I also looked at the confusion matrix to see exactly what mistakes the model made. I checked some wrong predictions by eye. Most mistakes were on blurry images or images with poor light.

One more honest point. My test set comes from the same dataset as my training set. So the real-world performance, with random phone photos, may be lower. I explain this in the limitations.

Part 10: Building the Telegram bot (2.5 min)

Now the second half of the project: the bot.

Creating the bot. I opened Telegram and searched for **BotFather**. BotFather is the official Telegram bot that creates other bots. I sent the command `/newbot`. I gave my bot a name and a username. BotFather then gave me a **token**.

This token is like a password for the bot. Anyone who has it can control the bot. So I did not write it openly in my code or share it publicly. I stored it as a secret variable.

Writing the bot code. I used Python with the [python-telegram-bot] library, which talks to the **Telegram Bot API**.

My bot has these handlers.

- A **start command**. When the user types /start, the bot says hello and explains how to use it.
- A **photo handler**. This runs when the user sends a photo.

Here is what the photo handler does, step by step.

1. It takes the photo from the message. Telegram sends several sizes, so I take the largest one.
2. It downloads the photo.
3. It reads the photo using **OpenCV**, converts BGR to RGB, resizes to 224 by 224, and divides by 255. These are the same steps as in training. This is very important, because the preprocessing must match.
4. It passes the image to the model using `model.predict`.
5. It reads the probability. If the value is above 0.5, the label is cancerous, and the confidence is that value. If it is below 0.5, the label is non-cancerous, and the confidence is 1 minus that value.
6. It sends the result back to the user as a message, with the confidence score and a short note: "This is not a medical diagnosis. Please consult a doctor."

I load the model **only once**, when the bot starts. If I loaded it for every image, the replies would be very slow.

The bot uses **polling**. This means my code keeps asking Telegram, "Are there any new messages?" This is simple and good for a project.

Part 11: Live demo walk-through (1 min)

Let me explain how a demo looks.

I open Telegram and start my bot. I type /start. The bot says welcome and asks me to send a clear photo of the mouth.

I send an image. In about [two to three] seconds, the bot replies. For example: "Result: Non-cancerous. Confidence: [92] percent. Please note this is not a medical diagnosis."

Then I send a cancerous sample image from the test set. The bot replies: "Result: Cancerous. Confidence: [88] percent. Please consult a doctor."

So the user does not need to know anything about AI. They just send a photo.

Part 12: Challenges and how I solved them (1.5 min)

I faced a few problems.

Challenge 1: Small dataset and overfitting. The model was doing very well on training data but not on validation data. I solved this with data augmentation, dropout, transfer learning, and early stopping.

Challenge 2: Wrong colours. At first, my bot gave different results from my notebook for the same image. After some debugging, I found the reason. OpenCV reads in BGR, and I forgot to

convert to RGB in the bot code. After I made the preprocessing the same in both places, the results matched.

Challenge 3: Colab disconnecting. My session sometimes stopped, and I lost progress. So I saved checkpoints and the final model to Google Drive.

Challenge 4: Slow replies. In the beginning, the bot was slow. I fixed this by loading the model only once and by using a light model, MobileNetV2.

Challenge 5: Image quality. Real photos can be blurry or dark. Augmentation helped, but this is still a limit.

Part 13: Limitations (1.5 min)

I want to be honest about the limitations.

First, this is a student project. It is **not a medical device**. It must not replace a doctor.

Second, the dataset is limited. It may not have all types of people, cameras, and lighting. The model may not work as well on real phone photos.

Third, the model answers for **any image**. If someone sends a photo of a dog or a table, the model will still give an answer. It does not know that the image is not a mouth.

Fourth, it only gives two outputs. It cannot tell the stage of cancer or the type.

Fifth, the bot runs from Google Colab. When my session stops, the bot stops. This is fine for a demo, but not for real use.

Sixth, privacy. Medical photos are sensitive. Telegram normal chats are encrypted between the app and the server, but they are not end-to-end encrypted. So a real product needs careful privacy rules.

Part 14: Future work (1 min)

If I continue this project, I will do these things.

- **Deploy the bot on a cloud server**, so it works all the time.
- **Add an image check**, so the bot first checks whether the image is really an oral image.
- **Add explainability** using Grad-CAM. This shows which part of the image the model looked at, so doctors can trust it more.
- **Train with more data**, and with data checked by doctors, to improve real-world accuracy.
- **Add more classes**, such as pre-cancerous lesions, and not just two.
- **Add local languages**, such as Telugu and Hindi, so more people can use it.

- **Protect user privacy**, by not storing the images, and by adding rate limits and input checks.

Part 15: Closing (1 min)

To summarise: I built an end-to-end system. I used a Kaggle image dataset and trained a deep learning model in Google Colab using TensorFlow and Keras. I used OpenCV for image preprocessing. Then I connected the model to a Telegram bot, so anyone can send a photo and get a result with a confidence score in seconds.

From this project, I learned the full process: collecting data, cleaning it, training, evaluating, and then **delivering the model to a real user**. I also learned that accuracy is not enough. We must think about recall, safety, privacy, and honest limits.

Thank you. I am happy to answer any questions.

FOLLOW-UP QUESTIONS THE INTERVIEWER MAY ASK

Practise these answers in your own words.

1. Why did you use a CNN?
CNNs are made for images. They find patterns like edges and textures by themselves, so I did not need to design features by hand.

2. Why transfer learning?
My dataset was small. A model pretrained on ImageNet already knows basic visual features, so I got better accuracy with less data and less training time.

3. What does the confidence score mean?
It is the model's probability for the predicted class. A score of 90 percent does not mean the person is 90 percent sure to have cancer. It only shows how strongly the model leans to one class. The model can be confidently wrong.

4. Why is recall important here?
Missing a real cancer case is more dangerous than a false alarm. So I want to catch as many real cases as possible.

5. How did you control overfitting?
Data augmentation, dropout, transfer learning with frozen layers, and early stopping. I also watched the validation loss.

6. What is the sigmoid function and why did you use it?
Sigmoid converts any number into a value between 0 and 1. I used it in the last layer for two-class output, so the result can be read as a probability.

7. Why binary cross-entropy?
Because there are only two classes. It punishes confident wrong predictions heavily, which helps the model learn.

8. Why do you divide pixels by 255?
To bring the values into the 0 to 1 range. Small, consistent numbers help the network train faster and more stably.

9. What does OpenCV do in your project?
It reads the image, converts the colour format, and resizes it, both in training and inside the bot.

10. How does the Telegram bot work?
BotFather gives a token. My Python code uses the token to talk to the Telegram Bot API. My code keeps polling for new messages. When a photo arrives, it downloads the photo, runs the model, and sends the reply.

11. What is the difference between polling and webhook?
In polling, my code keeps asking Telegram for new messages. In a webhook, Telegram sends the message to my server's URL when it arrives. Webhooks are better for production, and polling is simpler for development.

12. What if the user sends a non-oral image?
Right now the model will still give an answer, which is a limitation. A fix is to add a first model that checks whether the image is an oral image, or to reject results with low confidence.

13. How did you handle imbalanced data?
I first checked the class counts. If one class is bigger, I can use class weights, augmentation, or oversampling, and I judge the model by recall, precision, and F1 instead of accuracy.

14. How would you deploy this in real life?
I would put the model and bot on a cloud service like AWS, Render, or Hugging Face Spaces, use a webhook, keep the token in environment variables, and add logging and monitoring.

15. What is data leakage and how did you avoid it?
It is when test information gets into training, so scores look better than they really are. I split the data before any preprocessing choices and never used the test set while tuning.

16. What would you do differently?
I would collect more varied data, test on real phone photos, add Grad-CAM for explanation, and ask a doctor to review the results.

17. Is your system accurate enough for medical use?
No. It is a research and learning project. A real medical tool needs large, doctor-verified data, clinical testing, and approval from regulators.

Bonus: Security and privacy questions (useful for your Deloitte interview)

Even though the project is not about cyber security, the interviewer may test whether you think about security. These answers help.

18. How did you protect the bot token?
The token is like a password. I kept it in a secret variable, not in public code or GitHub. If it leaks, I can revoke it and get a new one from BotFather.

19. What security risks does your bot have?

- The token may leak.
- Users may send very large files or wrong file types, so I should check type and size.
- Someone may spam the bot, so I should add rate limits.
- Medical images are private, so I should not store them and should tell users how their data is used.
- Normal Telegram chats are not end-to-end encrypted.

20. Can someone attack the model?
Yes. A person could change an image slightly (an adversarial example) to make the model give a wrong answer. This is a known risk for image models. Defences include adversarial training, input checks, and human review for important cases.

E-Commerce Sales Data Analytics: Interview Script (about 24 minutes)

Time plan

Part	Topic	Time
1	Introduction	1 min
2	The problem and goals	1.5 min
3	Dataset	1.5 min
4	Tools used and why	1 min
5	Data cleaning	4 min
6	Feature creation	1.5 min
7	Exploratory data analysis	1 min
8	Sales trend analysis	2 min
9	Product and category analysis	1.5 min
10	Customer behaviour analysis	2.5 min
11	Inventory analysis	2 min
12	Business recommendations	1.5 min
13	Challenges	1 min
14	Limitations and future work	1.5 min
15	Closing	1 min
	Total	**about 24 min**

Speaking speed: about 130 words per minute. Do not memorise word by word. Memorise the order of the parts and the key idea of each one.

THE SCRIPT

Part 1: Introduction (1 min)

Good morning. My name is [Your Name]. I am a final-year B.Tech student in Artificial Intelligence and Data Science.

Today I will explain my project called **E-Commerce Sales Data Analytics**.

In this project, I took the sales data of an online store. I cleaned it, studied it, and made charts. My goal was to find useful answers for the business. For example, which products sell the most, which customers are valuable, and which items may go out of stock.

I used **Python, Pandas, NumPy, Matplotlib, and Seaborn**.

Part 2: The problem and goals (1.5 min)

Let me explain the problem first.

An online store collects a huge amount of data every day. It has orders, customers, products, prices, and stock. But raw data is just numbers in a table. It does not tell the owner what to do.

The owner has real questions. Which months have the highest sales? Which products make the most money? Who are the best customers? Which products are running out? Which products are not selling and wasting space?

So my project had three goals.

Goal 1. Understand the sales trends over time.

Goal 2. Understand customer behaviour.

Goal 3. Help with marketing and inventory decisions using these insights.

In short, I wanted to turn raw data into **clear and useful business decisions**.

Part 3: Dataset (1.5 min)

I used an e-commerce sales dataset from [Kaggle]. It has around [50,000] records, covering [12 months].

Each row is one order line. The main columns are order ID, order date, customer ID, product name, category, quantity, unit price, region, and stock level.

At the start, I did three simple checks in Pandas.

- `df.head()` to see the first rows.
- `df.info()` to see column names, data types, and missing values.
- `df.describe()` to see the minimum, maximum, and average of number columns.

This first look told me the data was messy. Some values were missing. Some dates were stored as text. And there were duplicate rows. So I needed to clean it before any analysis.

Part 4: Tools used and why (1 min)

I used four main libraries.

Pandas is for loading, cleaning, and analysing table data. It is the main tool of the project.

NumPy is for fast number calculations, like conditions and averages on large columns.

Matplotlib is for drawing charts like lines, bars, and histograms.

Seaborn is built on Matplotlib. It makes good-looking statistical charts, like heatmaps and box plots, with less code.

I chose Python because it is free, simple, and widely used in industry for data work.

Part 5: Data cleaning (4 min)

I spent a lot of time on cleaning, because bad data gives wrong insights. There is a saying: **"garbage in, garbage out."**

I will explain each problem and what I did.

Problem 1: Missing values.
I used `df.isnull().sum()` to count missing values in every column.

- For **numeric columns** like price, I filled missing values with the **median**. I chose the median, because it is not affected by very large or very small values.
- For **category columns** like region, I filled with the most common value, or with "Unknown".
- If the **customer ID or order ID** was missing, I removed that row, because the row cannot be used.

I did not delete everything blindly. I first checked how many values were missing. If only a few are missing, deleting is fine. If many are missing, deleting would lose useful data.

Problem 2: Duplicate rows.
Some orders appeared twice. This would count the same sale two times and make the sales look higher. I used `df.duplicated().sum()` to count them and `df.drop_duplicates()` to remove them. I was careful to compare using order ID and product, not just the full row.

Problem 3: Wrong data types.
The order date was saved as text. I converted it using `pd.to_datetime()`. After this, I could easily take the month, day, and weekday. I also checked that price and quantity were numbers, not text.

Problem 4: Wrong or impossible values.
I found some rows with negative quantity or zero price. I checked them. Some were returns, and some were errors. I kept the genuine returns in a separate column, and removed the clear errors.

Problem 5: Outliers.
I used a **box plot** and the **IQR method** to find unusually large values. For example, one order had a quantity of 5,000, while most orders had 1 to 5.

I did not delete outliers automatically. I checked each one. If it was a typing mistake, I removed it. If it was a real bulk order, I kept it. A real outlier may be an important business customer.

Problem 6: Inconsistent text.
The same category was written in different ways, like "electronics", "Electronics", and "ELECTRONICS ". I fixed it using `str.strip()` and `str.title()`, so they all became one value.

After cleaning, I checked again. I confirmed that there were no unwanted missing values or duplicates and that all types were correct. I kept a copy of the raw data, so I could always go back.

Part 6: Feature creation (1.5 min)

After cleaning, I created new columns to make the analysis easier. This is called **feature engineering**.

- **Total sales:** quantity multiplied by price. This is the revenue for each row.
- **Month, year, weekday, and hour:** taken from the order date, to study time patterns.
- **Order value:** the total amount of each order.
- **Customer-level numbers:** for each customer, I calculated total spending, number of orders, and the date of the last order.

These new columns turned the raw data into something I could ask questions of.

Part 7: Exploratory data analysis, EDA (1 min)

Now I started EDA, which means exploring the data with numbers and charts to find patterns.

First I looked at the overall picture: total sales, total number of orders, number of customers, and the average order value.

Then I looked at each column on its own, using histograms for numbers and count plots for categories. After that, I compared columns with each other, using bar charts, line charts, scatter plots, and a correlation heatmap.

I always gave each chart a title, axis labels, and clear colours, so a non-technical manager can understand it quickly.

Part 8: Sales trend analysis (2 min)

My first question was: **how do sales change over time?**

I used `groupby` on the month and added the total sales. Then I drew a **line chart** with Matplotlib.

What I found:

- Sales were highest in [October to December]. This matches the festival and holiday season.
- Sales were lowest in [February and March].
- I also found that [weekends] had more orders than weekdays.
- The busiest time of the day was [8 PM to 11 PM], when people shop after work.

I also calculated the **month-on-month growth rate**, to see if the business was growing or falling.

Why this matters: If the business knows when sales rise, it can plan stock and run ads before the busy period. In slow months, it can offer discounts to keep sales moving.

Part 9: Product and category analysis (1.5 min)

My second question was: **which products and categories make the most money?**

I grouped the data by category and by product, and I took the total sales and quantity. Then I drew **bar charts** of the top 10.

What I found:

- The top [3] categories were responsible for about [60 percent] of the revenue.
- A small number of products gave most of the revenue. This is similar to the **80/20 rule**, where a small part of the products gives most of the income.
- Some products sold a lot in quantity, but had a low price, so the revenue was small.
- Some products had low sales and a high return rate.

Why this matters: The business can focus on its best products for marketing, and review the weak ones.

Part 10: Customer behaviour analysis (2.5 min)

My third question was: **who are the customers and how do they behave?**

First, I found the **top customers by total spending**. A small group of customers gave a big share of the revenue.

Then I separated **new customers** and **repeat customers**. I found that [about 30 percent] of customers bought more than once, and that they spent much more in total.

I also calculated the **average order value** and checked how it changed by region.

For deeper analysis, I used a simple form of **RFM analysis**. RFM stands for three things.

- **Recency:** how recently did the customer buy?
- **Frequency:** how often do they buy?
- **Monetary:** how much money do they spend?

Using these three, I put customers into groups like these.

- **Loyal and high-value customers:** buy often and spend a lot.
- **New customers:** just started.
- **At-risk customers:** used to buy, but have not bought for a long time.

I visualised the segments with a bar chart and a heatmap.

Why this matters:

- Loyal customers can get rewards or early access to sales.
- At-risk customers can get a "We miss you" email with a discount.
- New customers can get a welcome offer to encourage a second purchase.

This is how data helps **marketing**: the right message goes to the right group.

Part 11: Inventory analysis (2 min)

My fourth question was: **how can we manage the stock better?**

Stock problems cost money in two ways. If a product is **out of stock**, the business loses a sale. If a product is **overstocked**, money is stuck in unsold goods.

I did three checks.

Check 1: Fast-moving and slow-moving products.
I calculated the average sales per month of each product, and compared this with the current stock. Fast-moving products need regular restocking. Slow-moving products need fewer orders or a discount.

Check 2: Possible stock-out risk.
I estimated how many days the stock will last. I divided the current stock by the average daily sales. Products with only a few days left were marked as high risk.

Check 3: Seasonal demand.
From the trend analysis, I found which products sell more in which months. So the store can order more stock before those months.

I showed the results with bar charts and a table of products that need urgent restocking.

Why this matters: Good inventory planning reduces lost sales and reduces the cost of storing unsold products.

Part 12: Business recommendations (1.5 min)

An analysis is only useful if it leads to action. So I wrote clear recommendations.

For marketing:

- Increase ads before the [October to December] peak.
- Send special offers to at-risk customers.
- Reward the top customers with a loyalty programme.
- Promote the top categories and bundle slow products with popular ones.

For inventory:

- Restock fast-moving products before they run out.
- Reduce orders of slow-moving products.
- Plan extra stock before the busy season.

For the business overall:

- Check products with a high return rate and find out why.

I presented these in simple language, so a manager without technical knowledge can act on them.

Part 13: Challenges (1 min)

I faced a few problems.

Messy data. Cleaning took most of my time. I learned to check the data carefully first, instead of rushing to charts.

Deciding what to do with outliers. At first, I wanted to delete all of them. But some were real bulk orders. I learned to check before removing.

Too many charts. In the beginning, I made many charts that did not answer any question. Then I started with a business question first, and made only the charts that answer it.

Slow code on a large file. Loops in Python were slow. I used Pandas and NumPy vectorised operations, which are much faster.

Part 14: Limitations and future work (1.5 min)

I want to be honest about the limits.

- The data covers only [one year], so I cannot be fully sure about yearly seasonality.
- This is **descriptive analysis**. It tells us what happened and where, but it does not predict the future.
- Correlation is not causation. If two things move together, it does not prove that one causes the other.
- I did not have data about marketing spend, so I could not measure which campaign worked.

Future work:

- Build a **sales forecasting model** using time series methods like ARIMA, Prophet, or LSTM.
- Build a **customer segmentation model** using K-Means clustering.
- Build a **recommendation system** to suggest products.
- Predict **customer churn**, meaning who is likely to stop buying.
- Create an **interactive dashboard** in Power BI, Tableau, or Streamlit, which updates automatically.
- Do **A/B testing** to check if a discount really increases sales.

Part 15: Closing (1 min)

To summarise: I took raw e-commerce data, cleaned it, created new features, and explored it with Pandas, NumPy, Matplotlib, and Seaborn. I found patterns in sales trends, products, customers, and inventory, and I turned them into clear recommendations for marketing and stock planning.

From this project, I learned the full data analysis process, from raw data to business decisions. I also learned that cleaning the data is the most important step, and that charts are only useful when they answer a real question.

Thank you. I am happy to answer any questions.

FOLLOW-UP QUESTIONS THE INTERVIEWER MAY ASK

Practise these answers in your own words.

1. How did you handle missing values?
First I checked how many were missing and why. For numbers, I used the median. For categories, I used the most common value or "Unknown". I removed rows only when key fields like customer ID were missing.

2. Why median and not mean?
The mean is pulled by very large or small values (outliers). The median is the middle value, so it is more stable.

3. How did you detect outliers?
With box plots and the IQR method. Values below Q1 minus 1.5 times IQR, or above Q3 plus 1.5 times IQR, are marked as outliers. Then I checked them by hand.

4. What is the difference between `loc` and `iloc`?
`loc` selects by label or condition. `iloc` selects by position number.

5. What is `groupby`?
It splits the data into groups, applies a calculation like sum or mean to each group, and combines the results. I used it for monthly sales and sales per category.

6. What is the difference between Matplotlib and Seaborn?
Matplotlib gives full control over charts. Seaborn is built on Matplotlib and makes good-looking statistical charts with less code.

7. How did you choose which chart to use?
Line chart for trends over time, bar chart for comparing categories, histogram for distribution, scatter plot for relation between two numbers, heatmap for correlations.

8. What is RFM analysis?
It groups customers by Recency (last purchase), Frequency (how often), and Monetary value (how much). It helps to find loyal, new, and at-risk customers.

9. What was your most important insight?
Pick your real one. Example: "A small group of repeat customers produced a big part of the revenue, so keeping them happy matters more than finding new ones."

10. How did the project help the business?
It gave clear actions: when to run ads, which customers to target, and which products to restock or discount.

11. How would you predict future sales?
I would use time series forecasting, such as ARIMA, Prophet, or LSTM, and check the error with MAE or RMSE on the most recent months.

12. What is correlation vs causation?
Correlation means two things move together. Causation means one causes the other. A strong correlation alone does not prove causation. We need controlled tests like A/B testing.

13. How did you check that your cleaning did not damage the data?
I kept a copy of the raw data, compared row counts and totals before and after cleaning, and checked that key numbers still made sense.

14. How would you handle a very large dataset that does not fit in memory?
Read the file in chunks using `chunksize`, choose smaller data types, use only the needed columns, or use tools like Spark or a database.

15. What would you do differently?
Add more months of data, forecast the sales, and build an interactive dashboard.

Bonus: Data privacy and security questions (useful for your Deloitte interview)

An interviewer for a cyber security role may test whether you think about data protection even in an analytics project.

16. Did the dataset have personal data? How would you protect it?
Customer data like names, phone numbers, emails, and addresses is personal data. In real projects, I would remove or mask it, use hashed customer IDs, give access only to the people who need it, and follow rules like GDPR and India's DPDP Act.

17. What is data anonymisation?
Removing or hiding details so that a person cannot be identified. Methods include masking, hashing, and generalising (for example, using an age group instead of the exact age).

18. What can go wrong if sales data leaks?
Customer privacy is lost, and competitors may see prices and sales, which causes legal and business damage. So data needs encryption, access control, and logging.

19. How can analytics help in security?
The same skills can find fraud and unusual behaviour. For example, a sudden spike in orders from one IP address, or many failed payments, can be an attack. I can use `groupby` and charts to spot such patterns.

20. How do you make sure your analysis is correct and trustworthy?
Check the data quality, keep the steps in a notebook so they can be repeated, compare totals with the source, and have someone review the results.

Quick code lines to remember

You may be asked to write small pieces of code. Practise these.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns

df = pd.read_csv("sales.csv")
df.info(); df.describe()
df.isnull().sum()
df = df.drop_duplicates()
df["order_date"] = pd.to_datetime(df["order_date"])
df["total_sales"] = df["quantity"] * df["price"]
df["month"] = df["order_date"].dt.month

monthly = df.groupby("month")["total_sales"].sum()
monthly.plot(kind="line", marker="o", title="Monthly Sales")
plt.show()

top = df.groupby("category")["total_sales"].sum().sort_values(ascending=False).head(10)
sns.barplot(x=top.values, y=top.index)
plt.show()

sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.show()
```